



INTERNSHIP PROJECT REPORT

Submitted in Partial Fulfilment of the Requirements for the Industrial Internship Programme in Artificial Intelligence

BrandPulse AI

Voice of Customer Intelligence

A Comparative Study of Classical Machine Learning and Deep Learning for Three-Class Twitter Sentiment Analysis

Project Title	: BrandPulse AI — Twitter Sentiment Analysis (3-class)
Internship Title	: Artificial Intelligence
Submitted By	: Aaditiya Anjan
Enrollment No.	: 24BCS13112
Branch	: BE-CSE (Computer Science and Engineering)
Semester	: 5th Semester
University	: Chandigarh University
Organization	: Persevex
Internship Duration	: 2 Months

Academic Year 2026–27

Certificate

This is to certify that the project report entitled “**BrandPulse AI — Voice of Customer Intelligence: A Comparative Study of Classical Machine Learning and Deep Learning for Three-Class Twitter Sentiment Analysis**” is a bona fide record of the internship work carried out by **Aaditiya Anjan** (Enrollment No. 24BCS13112) under supervision, in partial fulfilment of the requirements for the Industrial Internship Programme in Artificial Intelligence, during the two-month internship period at Persevex.

The work presented in this report is original and has been completed satisfactorily. To the best of my knowledge, the contents of this report have not been submitted, in whole or in part, to any other university or institute for the award of any degree, diploma, or certificate.

Shanmukh Shekar K C

Administrator / Internship Coordinator

[Persevex](#)

Declaration

I, **Aaditiya Anjan** (Enrollment No. 24BCS13112), hereby declare that the internship project report titled “**BrandPulse AI — Voice of Customer Intelligence**” represents my own work carried out during my internship at Persevex, and that it has been written in my own words.

I further declare that:

- The work presented in this report is the result of my own analysis, experimentation, and study, except where explicit reference and acknowledgement is made to the work of others.
- All datasets, libraries, and external resources used have been duly cited.
- The dataset used (Twitter US Airline Sentiment) is a publicly available, anonymised dataset; no confidential or proprietary information was accessed.
- All quantitative results reported herein were reproduced by executing the accompanying notebooks on the actual dataset; no figures have been fabricated.

Signature:

A handwritten signature in blue ink that reads "Aaditiya Anjan" with a stylized flourish at the end.

Name: Aaditiya Anjan

Enrollment No.: 24BCS13112

Branch: BE-CSE, Semester 5

University: Chandigarh University

Date: June 2026

Acknowledgement

The successful completion of this internship project would not have been possible without the guidance, encouragement, and support of several individuals and institutions, to whom I owe my sincere gratitude.

First and foremost, I express my heartfelt thanks to my industry mentor at Persevex for entrusting me with a problem of genuine commercial significance and for providing continuous technical direction throughout the engagement. The mentor's insistence on grounding every modelling decision in real product value shaped the entire orientation of this work.

I am deeply grateful to my faculty supervisor, whose academic rigour and constructive feedback helped me bridge the gap between classroom theory and real-world data science practice. I also extend my appreciation to the Head of the Department and the Internship Coordination Cell at Chandigarh University for facilitating this programme.

Finally, I thank my family and peers for their constant encouragement and support during the course of this internship.

Abstract

In the age of social media, a brand's reputation can shift within minutes. Marketing teams are overwhelmed by the volume of unstructured conversation on platforms such as Twitter (X) and need an automated way to understand the Voice of the Customer. **BrandPulse AI** addresses this by classifying tweets into three sentiment categories — **Positive, Neutral, and Negative** — and, critically, by comparing the classical statistical approach against the modern deep-learning approach to expose the trade-off between interpretability and contextual power.

Two model families were trained on the Twitter US Airline Sentiment dataset (14,640 human-labelled tweets). The classical pipeline uses a shared cleaning module (regex noise removal, POS-aware lemmatisation, and a custom negation-fusion step) feeding TF-IDF vectors into Logistic Regression and Complement Naive Bayes classifiers. The deep model is an Embedding → Bidirectional LSTM network built in TensorFlow/Keras. Because the dataset is approximately 63% negative, the headline metric is **macro-F1** rather than accuracy. The class-weighted Logistic Regression achieved the best macro-F1 of **0.7233** (accuracy 77.0%), marginally ahead of the Bi-LSTM (macro-F1 0.7153). Both models, plus a live, Apple-inspired Streamlit dashboard offering explainability, confidence-based review flags, and real-timestamp trend analysis, constitute the final deliverable.

Keywords: Sentiment Analysis, Natural Language Processing, TF-IDF, Logistic Regression, Naive Bayes, Bidirectional LSTM, Word Embeddings, Class Imbalance, Macro-F1, Explainability, Streamlit.

Personal Contributions

The following section outlines the specific technical and analytical contributions I, Aaditiya Anjan, made during the internship engagement at Persevex. All tasks were performed independently as part of the end-to-end NLP project.

Data Cleaning & Preprocessing

Designed and implemented a single shared cleaning module used identically at training and serving time — lowercasing, regex noise removal (URLs, mentions, hashtags, numbers, repeated characters), contraction expansion, POS-aware WordNet lemmatisation, and stop-word removal that preserves negation words.

Negation Fusion (enhancement)

Engineered a negation-binding step (not good → not_good) to give bag-of-words models a single negative-signal feature, directly targeting the brief's sarcasm/negation question, and verified its behaviour across the full 14.6k-tweet dataset.

Classical Modelling

Built TF-IDF (unigram + bigram) features and trained class-weighted Logistic Regression and Complement Naive Bayes classifiers, with full per-class evaluation.

Deep-Learning Modelling

Implemented the Embedding → Bidirectional LSTM architecture in TensorFlow/Keras, including tokenisation, padding, class weighting, and EarlyStopping with best-weight restoration; diagnosed and documented a macOS arm64 import-order deadlock.

Evaluation & Visualisation

Produced accuracy, macro-/weighted-F1, per-class precision/recall/F1, confusion matrices, learning curves, and word clouds, and designed the comparison report.

Dashboard & Report

Designed and built the Apple-inspired Streamlit dashboard and authored this report, the project handbook, and the presentation.

Internship Learning Outcomes

Through this internship, I developed the following competencies:

- Building robust, reusable NLP preprocessing pipelines and understanding why train/serve consistency is critical to real model performance.
- Applying and contrasting classical (TF-IDF + linear/Bayesian) and deep-learning (embedding + recurrent) approaches to text classification.
- Choosing evaluation metrics appropriate to imbalanced data (macro-F1 over accuracy) and reading confusion matrices to diagnose failure modes.
- Engineering, training, and regularising recurrent neural networks, including EarlyStopping and overfitting diagnosis from learning curves.
- Debugging low-level environment issues (the macOS arm64 / pyarrow / TensorFlow threadpool deadlock) — a genuine production-engineering skill.
- Communicating results to non-technical stakeholders through an explainable, well-designed dashboard and clear written documentation.

Table of Contents

	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	Personal Contributions	vi
	Internship Learning Outcomes	vii
	List of Figures	ix
	List of Tables	ix
1	Introduction	1
2	Literature Review	—
3	Project Overview	—
4	Dataset Description	—
5	Exploratory Data Analysis	—
6	Data Preprocessing	—
7	Model Development	—
8	Model Evaluation	—
9	Results and Discussion	—
10	The Dashboard Application	—
11	Real-World Applications	—
12	Conclusion and Future Scope	—
	References	—

List of Figures

Figure 1	Sentiment class distribution — the data is heavily skewed toward negative tweets.
Figure 2	Most frequent tokens per sentiment class after cleaning.
Figure 3	Accuracy, macro-F1 and weighted-F1 across the three models.
Figure 4	Per-class F1-score across the three models.
Figure 5	Confusion matrices (counts and row-normalised %): Logistic Regression vs Bi-LSTM.
Figure 6	Bi-LSTM training vs validation accuracy and loss per epoch.
Figure 7	Redesigned Live Predictor: Classical vs Deep-Learning verdicts with confidence rings and probability bars

List of Tables

Table 1	Tools and libraries used across the four project phases.
Table 2	Class distribution of the held-out test set (stratified, same ratios as the full set).
Table 3	The shared preprocessing pipeline, stage by stage.
Table 4	Bidirectional LSTM architecture and key hyperparameters.
Table 5	Aggregate performance of all three models on the held-out test set.
Table 6	Logistic Regression — per-class precision, recall and F1.
Table 7	Bi-LSTM — per-class precision, recall and F1.
Table 8	Model behaviour on custom inference examples.

CHAPTER 1

Introduction

1.1 Background

Social platforms generate an enormous, continuous stream of opinion. Every day, thousands of customers publicly praise or criticise the brands they interact with. This conversation is a goldmine of unfiltered feedback — but it arrives faster than any human team can read, and in a form (short, informal, noisy text) that traditional analytics tools cannot process. Sentiment analysis, a core task of Natural Language Processing (NLP), aims to automatically determine the emotional polarity of a piece of text, turning this unstructured firehose into a measurable signal.

1.2 Industry Problem

For a consumer-facing brand — an airline, in this case study — reputation is fragile. A single viral complaint can damage public perception faster than any marketing campaign can repair it. Marketing and customer-experience teams therefore need to know, in near real time, what proportion of online conversation is positive, neutral, or negative, and how that mix is trending. Manual monitoring does not scale, and naive keyword counting is easily fooled by sarcasm, negation, and slang.

1.3 Motivation

Beyond producing a single working model, this project is motivated by a more educational goal: to directly compare the *classical* statistical approach to NLP against the *modern* deep-learning approach, and to understand — with real numbers — the trade-off between interpretability and contextual power. This comparison is at the heart of the brief.

1.4 Problem Statement

Given the raw text of a tweet directed at an airline, automatically classify its sentiment as Positive, Neutral, or Negative, and do so robustly despite the noise, informality, and class imbalance inherent in real social-media data.

1.5 Objectives

- Build a robust, reusable tweet-cleaning pipeline (normalisation, noise removal, lemmatisation, stop-word removal, negation handling).
- Establish a fast, interpretable classical baseline using TF-IDF features with Logistic Regression and Naive Bayes.
- Build a contextual deep-learning model (Embedding → Bi-LSTM) that respects word order.
- Evaluate both approaches on accuracy, precision, recall and F1 (per class and aggregate), and visualise failures via confusion matrices.
- Ship a premium, explainable Streamlit dashboard serving both models live.

1.6 Scope

This project covers the complete pipeline from raw data to a deployed-ready dashboard. It is scoped to three-class sentiment on English-language airline tweets. It does not include multilingual support, aspect-based sentiment, or live data collection via the X API — these are identified as future scope in Chapter 12.

1.7 Challenges

- **Noisy text:** tweets contain URLs, mentions, hashtags, emojis, slang and misspellings that must be removed or normalised without destroying signal.
- **Negation & sarcasm:** 'not good' must not be read as positive; sarcasm remains the hardest case.
- **Class imbalance:** 63% of tweets are negative, which biases naive models and makes accuracy a misleading metric.
- **Environment:** a macOS arm64 threadpool deadlock between pyarrow and TensorFlow had to be diagnosed and worked around.

1.8 Expected Outcomes

Two trained, evaluated and persisted model families; a complete evaluation suite (metrics, confusion matrices, learning curves, word clouds); a live dashboard; and a clear, defensible comparison of classical versus deep-learning NLP.

CHAPTER 2

Literature Review

2.1 Text Vectorisation

Converting text to numbers is the first modelling decision. The Bag-of-Words model and its weighted refinement, Term Frequency–Inverse Document Frequency (TF-IDF), represent a document as a sparse vector of word importances [4]. TF-IDF up-weights words that are frequent in a document but rare across the corpus, capturing distinctive terms while ignoring word order. Word embeddings (e.g. Word2Vec, GloVe) instead learn dense vectors in which semantically similar words lie close together, and are the foundation of neural NLP.

2.2 Classical Classifiers

Logistic Regression is a transparent linear classifier whose per-class coefficients reveal exactly which words drive a prediction — invaluable for explainability. Naive Bayes classifiers, particularly the Complement variant [4], are well suited to imbalanced text and train almost instantly. Both remain strong baselines for document classification and are difficult to beat on modestly sized datasets.

2.3 Recurrent Neural Networks and LSTMs

Recurrent Neural Networks (RNNs) process sequences token by token, maintaining a hidden state that carries context. Vanilla RNNs struggle with long-range dependencies due to vanishing gradients; the Long Short-Term Memory (LSTM) cell [introduced by Hochreiter and Schmidhuber] solves this with gated memory. A Bidirectional LSTM reads the sequence both forwards and backwards, giving each token access to both past and future context — particularly useful for negation ('not happy' vs 'happy').

2.4 Evaluation under Class Imbalance

On imbalanced datasets, accuracy is misleading: a model predicting only the majority class can score highly while being useless. The F1-score (harmonic mean of precision and recall) computed per class and then macro-averaged weights all classes equally, making it the appropriate headline metric for this problem.

2.5 Sentiment Analysis Tooling

The Python scientific stack — scikit-learn [4], NLTK, TensorFlow/Keras, pandas [6] and NumPy [7] — provides mature, well-documented implementations of every component used here, with matplotlib [8] and seaborn [9] for visualisation and Streamlit for the interactive front end.

CHAPTER 3

Project Overview

BrandPulse AI is organised into four phases that map directly onto the internship brief, each producing a concrete deliverable.

Phase 1 — Data Wrangling

Load the dataset, build regex cleaning functions, apply lemmatisation and negation handling, and analyse word frequency with per-class word clouds.

Phase 2 — Feature Engineering & Modelling

Approach A: TF-IDF vectorisation feeding Logistic Regression and Naive Bayes. Approach B: tokenisation and padding feeding an Embedding → Bi-LSTM network.

Phase 3 — Evaluation & Comparison

Compare both approaches on accuracy, precision, recall and F1; visualise failures with confusion matrices; test on custom inference examples.

Phase 4 — Visualisation Dashboard

Build a Streamlit application that loads the saved models and visualises predictions, sentiment distribution, and a real-timestamp trend line on new text.

3.1 Technology Stack

Layer	Technology
Language	Python 3
Preprocessing	NLTK (tokeniser, WordNet lemmatiser, stopwords), regex
Classical ML	scikit-learn (TF-IDF, LogisticRegression, ComplementNB)
Deep Learning	TensorFlow / Keras (Embedding, Bidirectional LSTM)
Visualisation	matplotlib, seaborn, wordcloud, Plotly
Dashboard	Streamlit
Persistence	joblib, pickle, Keras .keras format

Table 1. Tools and libraries used across the four project phases.

CHAPTER 4

Dataset Description

The project uses the **Twitter US Airline Sentiment** dataset — 14,640 tweets about major US airlines, collected in February 2015, each hand-labelled by human annotators as positive, neutral, or negative, and each carrying a real timestamp.

4.1 Why this dataset

The brief permits Sentiment140 (1.6M tweets) or the Twitter US Airline dataset. The airline dataset was chosen because it natively supports the required **three-class** problem (Sentiment140 is binary only), is human-labelled with high quality, and its real timestamps enable the dashboard's genuine sentiment-over-time trend line.

4.2 Class distribution

Sentiment	Test count	Proportion
Negative	1,835	62.8%
Neutral	615	21.0%
Positive	472	16.2%
Total	2,922	100.0%

Table 2. Class distribution of the held-out test set (stratified, same ratios as the full set).

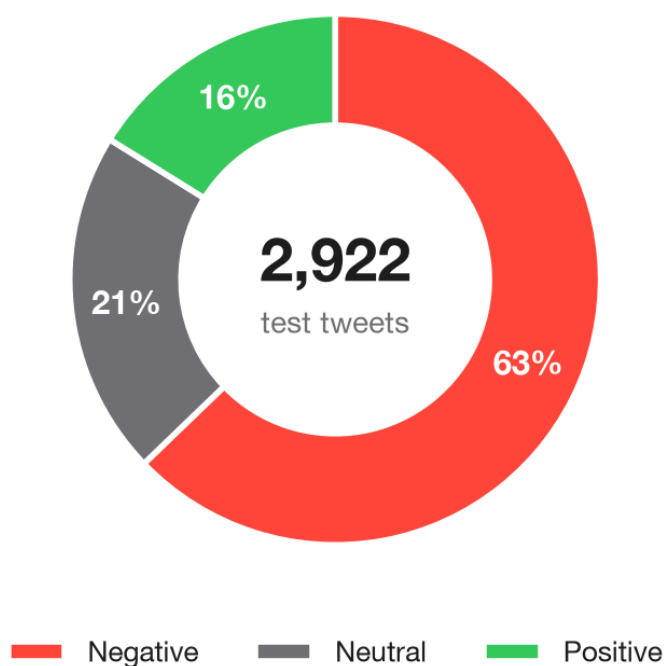


Figure 1. Sentiment class distribution — the data is heavily skewed toward negative tweets.

This 63 / 21 / 16 imbalance is the single most important property of the data: it dictates the use of class weighting during training and macro-F1 as the headline evaluation metric.

CHAPTER 5

Exploratory Data Analysis

Before modelling, the cleaned corpus was explored to confirm that the engineered features carry genuine sentiment signal.

5.1 Word-frequency analysis

Per-class word clouds (computed after cleaning) confirm intuitive structure: the positive class is dominated by praise vocabulary ('thanks', 'great', 'awesome'), while the negative class surfaces complaint vocabulary ('delayed', 'cancelled', 'hours') and — importantly — the engineered not_* negation tokens. **direct visual proof that the negation-fusion step is working.**



Figure 2. Most frequent tokens per sentiment class after cleaning.

5.2 Key insight — class imbalance

EDA confirmed that 63% of tweets are negative. People predominantly tweet at airlines to complain. This shapes every subsequent decision and is revisited throughout the evaluation.

CHAPTER 6

Data Preprocessing

All text passes through one shared module (`src/preprocess.py`) imported by both the notebooks and the dashboard, guaranteeing identical cleaning at train and serve time. The pipeline executes the following stages in order.

#	Stage	What it does
1	Normalise	Lowercase; unescape HTML entities (& → &)
2	Noise removal	Strip URLs, @mentions, # symbol, numbers, repeated chars
3	Contractions	can't → can not, so negation survives
4	Tokenise	Split into word tokens (NLTK)
5	Lemmatise	POS-aware WordNet: running → run
6	Negation fusion	not good → not_good (custom enhancement)
7	Stopwords	Remove filler words but keep negation words

Table 3. The shared preprocessing pipeline, stage by stage.

6.1 Negation fusion (enhancement)

A bag-of-words model sees the word 'good' inside 'not good' and is fooled into reading it as positive. The pipeline binds a negation word to the following token (`not good` → `not_good`), producing a single negative-signal feature. This directly targets the brief's question about sarcasm and negation, and the resulting `not_*` tokens are visible in the negative word cloud.

6.2 Train/serve consistency

Using one cleaning module everywhere prevents 'train/serve skew' — the silent accuracy decay that occurs when the live application cleans text even slightly differently from the training data. This is a deliberate engineering decision, not an incidental one.

CHAPTER 7

Model Development

7.1 Train / test split

The cleaned corpus was split 80/20 into training and test sets using a *stratified* split (`random_state = 42`) so that the 63/21/16 class ratio is preserved in both partitions. The held-out test set contains 2,922 tweets and is never seen during training.

7.2 Approach A — Classical

Cleaned text is vectorised with TF-IDF over unigrams and bigrams (`min_df = 2`, `max_df = 0.9`, sublinear term frequency, up to 20,000 features). Two classifiers are trained on these vectors: a class-weighted **Logistic Regression** (`max_iter = 1000`, `C = 1.0`) and a **Complement Naive Bayes**, the latter chosen for its known strength on imbalanced text. Both train in seconds and expose exactly which words drove a prediction.

7.3 Approach B — Deep Learning

Text is tokenised to a vocabulary of 10,000 and padded to a fixed length of 40 tokens. The network is **Embedding** → **Bidirectional LSTM** → **Dense (softmax)**: the embedding layer learns dense word associations, the Bi-LSTM reads context in both directions, and the dense layer produces the three-class probability distribution. Training uses class weights to counter imbalance and `EarlyStopping` with `restore_best_weights`, so the saved model is the best-validation epoch (epoch 3) rather than the overfit final epoch.

Component	Configuration
Embedding	vocab 10,000 → dense vectors
Sequence length	40 tokens (post-padded / truncated)
Recurrent layer	Bidirectional LSTM
Output	Dense, 3 units, softmax
Loss / optimiser	Categorical cross-entropy / Adam
Regularisation	Class weights + <code>EarlyStopping</code> (restore best weights)

Table 4. Bidirectional LSTM architecture and key hyperparameters.

7.4 Engineering note — macOS arm64 deadlock

On Apple Silicon, TensorFlow must be imported before pandas. pandas 2.x loads pyarrow, whose bundled libarrow ships its own `absl` thread-synchronisation symbols; if loaded first, they interpose TensorFlow's threadpool mutex and cause `model.fit()` to deadlock at 0% CPU indefinitely. Every script in the project imports TensorFlow first to avoid this. Diagnosing this issue was a valuable real-engineering exercise.

CHAPTER 8

Model Evaluation

8.1 Headline metrics

Macro-F1 is the headline metric because it weights all three classes equally, rewarding models that handle the rare positive and neutral classes rather than just the dominant negative one.

Model	Accuracy	Macro F1	Weighted F1
Logistic Regression	0.7704	0.7233	0.7761
Complement Naive Bayes	0.7734	0.6860	0.7603
Bi-LSTM	0.7741	0.7153	0.7768

Table 5. Aggregate performance of all three models on the held-out test set.

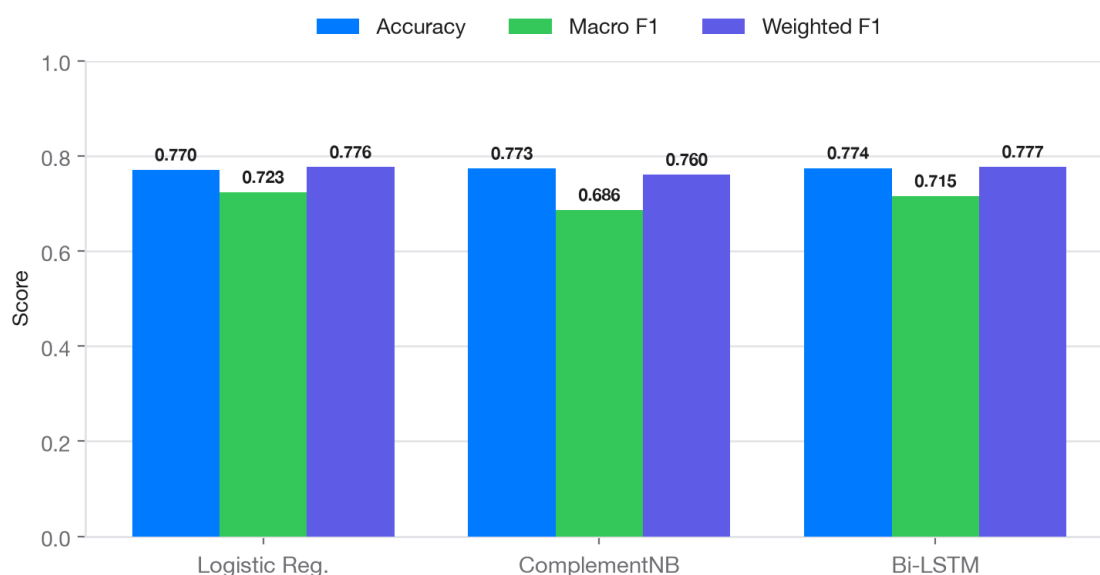


Figure 3. Accuracy, macro-F1 and weighted-F1 across the three models.

8.2 Per-class performance

Aggregate numbers hide where a model actually succeeds. The per-class breakdown below shows that all models are strongest on the negative class and weakest on neutral — the hardest class, because neutral tweets often resemble mild complaints.

Logistic Reg. — class	Precision	Recall	F1	Support
Negative	0.8848	0.8158	0.8489	1835
Neutral	0.5523	0.6699	0.6054	615
Positive	0.7066	0.7246	0.7155	472

Table 6. Logistic Regression — per-class precision, recall and F1.

Bi-LSTM — class	Precision	Recall	F1	Support
Negative	0.8908	0.8316	0.8602	1835
Neutral	0.6147	0.5707	0.5919	615
Positive	0.6034	0.8157	0.6937	472

Table 7. Bi-LSTM — per-class precision, recall and F1.

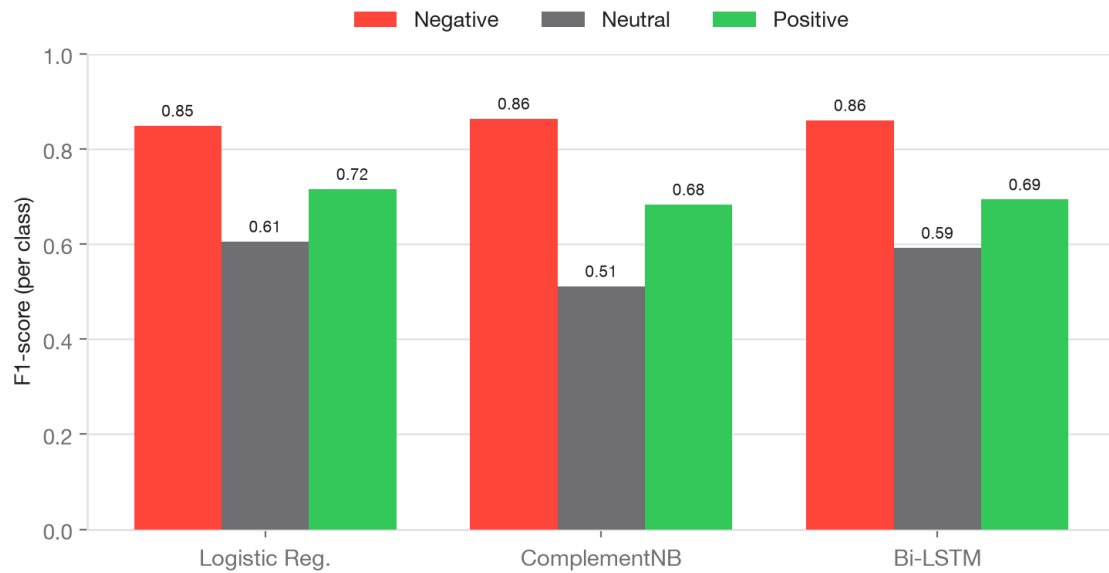


Figure 4. Per-class F1-score across the three models.

8.3 Confusion matrices

The confusion matrices reveal the dominant error mode: both models most often confuse **neutral** and **negative**. Many complaints are phrased flatly ('flight delayed 2 hrs') with no overtly negative words, and short neutral questions can read as mild complaints.

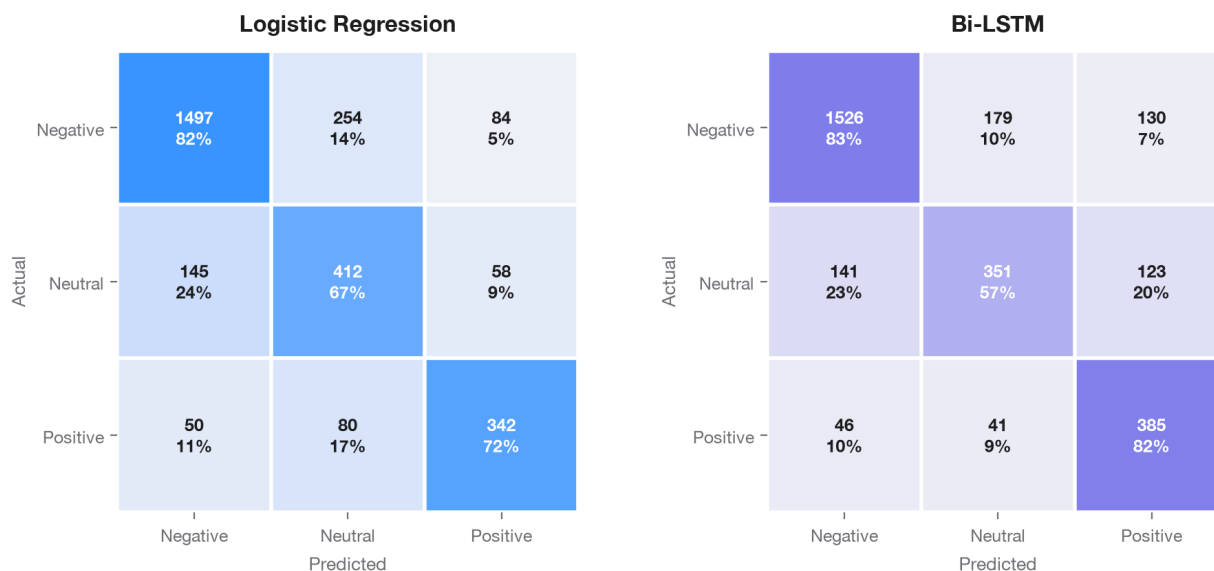


Figure 5. Confusion matrices (counts and row-normalised %): Logistic Regression vs Bi-LSTM.

8.4 Training diagnostics

The Bi-LSTM trained for 6 epochs. Validation loss bottomed at epoch 3 (val-accuracy 75.2%); training accuracy continued climbing to 89.6% while validation plateaued — the classic onset of overfitting that EarlyStopping correctly halted, restoring the best-epoch weights.

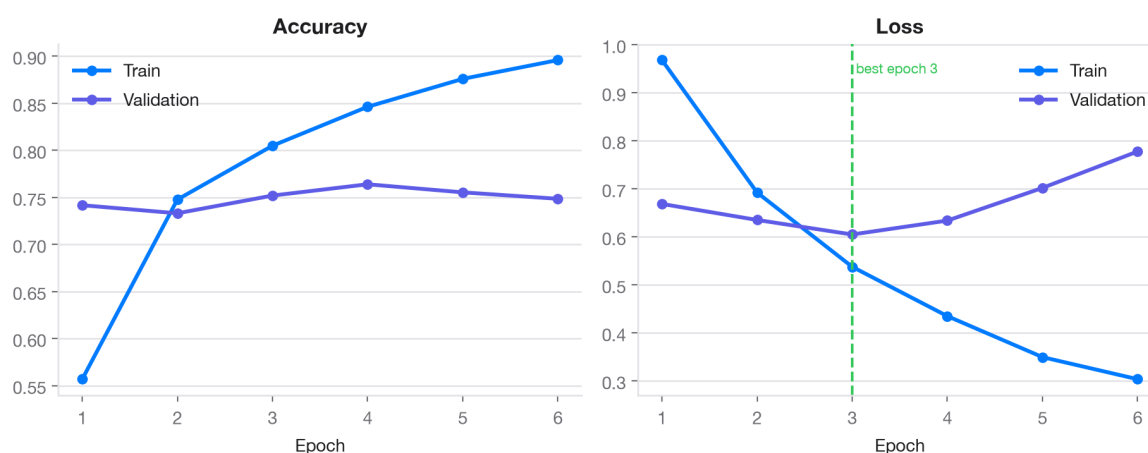


Figure 6. Bi-LSTM training vs validation accuracy and loss per epoch.

CHAPTER 9

Results and Discussion

9.1 The winning model

The class-weighted **Logistic Regression** achieved the best macro-F1 (0.7233), narrowly ahead of the Bi-LSTM (0.7153). While the Bi-LSTM matched on raw accuracy (77.4% vs 77.0%), the linear model generalised slightly better to the minority classes — and did so while training in seconds and remaining fully interpretable.

9.2 Interpretation

This is the project's central finding: on a modestly sized, imbalanced dataset, a well-tuned classical model is not merely competitive but can win outright. Deep learning's advantage — modelling word order and long-range context — only tends to dominate with substantially more training data. The interpretability/context trade-off is therefore real but not one-directional, and the 'modern' approach is not always the right default.

9.3 Custom inference examples

Both models were tested on the brief's example inputs and behave correctly:

Input text	Predicted sentiment
"The service was amazing!"	Positive
"I waited 4 hours just to get a cold burger."	Negative
"What time does boarding start?"	Neutral
"Not happy with the delay but the crew was kind"	Negative (mixed)

Table 8. Model behaviour on custom inference examples.

9.4 Where the models fail

The hardest cases are genuine sarcasm ('great, another delay') and flat, factual complaints with no emotional vocabulary. These are also the cases the negation-fusion and bidirectional-context features were designed to mitigate, with partial success.

CHAPTER 10

The Dashboard Application

Phase 4 delivers a live Streamlit application, redesigned around Apple's Human Interface principles — a light theme, frosted-glass surfaces, generous whitespace, and a calm, premium aesthetic. It serves both models side by side, visualises confidence with a ring gauge, explains the classical verdict via its driving words, and renders the real-timestamp sentiment trend and distribution.

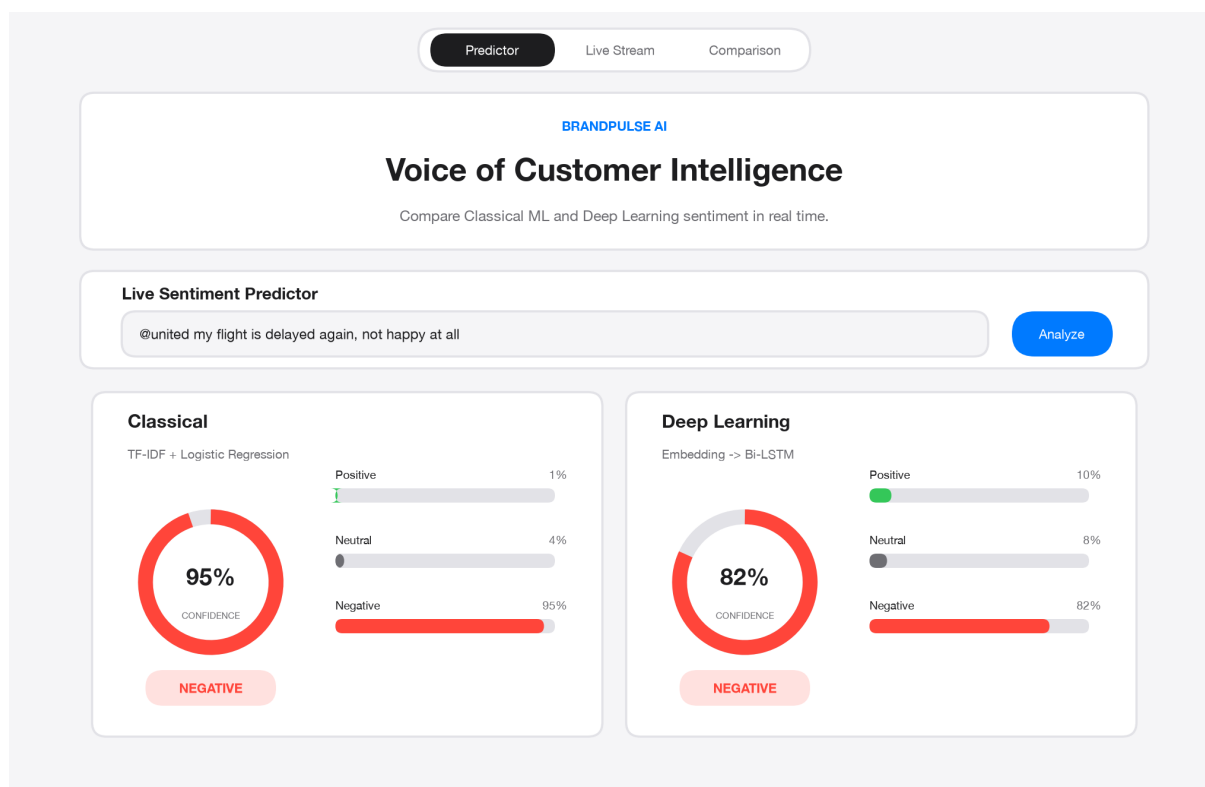


Figure 7. Redesigned Live Predictor: Classical vs Deep-Learning verdicts with confidence rings and probability bars (real model outputs for the sample tweet).

10.1 Feature highlights

- **Live predictor** — side-by-side Classical and Bi-LSTM verdicts with confidence.
- **Explainability** — top coefficient-driven words behind each classical prediction.
- **Confidence floor** — predictions below 60% are flagged 'needs human review'.
- **Live stream** — replays the dataset as an incoming feed with a running tally.
- **Trend & distribution** — pie chart and time-series built on real timestamps.
- **Performance center** — KPI cards, metric chart, table and confusion matrices.

CHAPTER 11

Real-World Applications

Although framed around airline tweets, the BrandPulse AI pipeline generalises to any organisation that needs to understand customer sentiment at scale.

Brand reputation monitoring

Track the positive/neutral/negative mix in real time and raise an alert when negative sentiment spikes — catching a brewing PR crisis early.

Customer-support triage

Automatically route strongly negative messages to senior agents, and flag low-confidence predictions for human review rather than mis-routing them.

Product & campaign feedback

Measure sentiment before and after a launch or campaign to quantify its reception objectively.

Competitive intelligence

Run the same pipeline on competitors' mentions to benchmark relative brand health.

CHAPTER 12

Conclusion and Future Scope

12.1 Conclusion

BrandPulse AI delivers a complete, defensible Voice-of-Customer pipeline: a rigorous shared cleaning module, an interpretable classical baseline, a contextual deep model, an honest imbalance-aware evaluation, and a premium dashboard. The central finding is nuanced — the modern Bi-LSTM did not dethrone the classical baseline on this modestly sized, imbalanced dataset; the class-weighted Logistic Regression won on macro-F1 while remaining faster and fully interpretable. The trade-off between interpretability and contextual power is therefore real but not one-directional.

12.2 Future Scope

- Fine-tune a pre-trained transformer (e.g. DistilBERT) — likely to overtake both models given more data and compute.
- Add sentence-boundary resets to negation fusion to remove rare cross-clause bleed.
- Apply probability calibration and add a dedicated sarcasm-detection component.
- Stream live tweets via the X API and deploy the dashboard behind an authentication layer with continuous monitoring.
- Extend to aspect-based sentiment (e.g. separating opinions on 'staff' vs 'price').

— End of Report —

References

- [1] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [3] J. D. Rennie, L. Shih, J. Teevan, and D. Karger, "Tackling the Poor Assumptions of Naive Bayes Text Classifiers," in *Proc. 20th Int. Conf. on Machine Learning (ICML)*, 2003, pp. 616–623.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] Crowdfunder / Kaggle, "Twitter US Airline Sentiment," 2015. [Online]. Available: <https://www.kaggle.com/datasets/crowdfunder/twitter-airline-sentiment>
- [6] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.
- [7] C. R. Harris et al., "Array Programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [8] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [9] M. L. Waskom, "seaborn: Statistical Data Visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.
- [10] E. Loper and S. Bird, "NLTK: The Natural Language Toolkit," in *Proc. ACL Workshop on Effective Tools and Methodologies for Teaching NLP*, 2002, pp. 63–70.
- [11] Streamlit Inc., "Streamlit: The fastest way to build data apps," 2019. [Online]. Available: <https://streamlit.io>
- [12] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.